Design and Implementation of a Wireless Telemetry Mesh Network

Objective

The objective of this experiment is to design and implement a Telemetry Mesh Network using a NodeMCU ESP8266 as the master node and two ESP32 microcontrollers as slave nodes, communicating wirelessly via the ESP-NOW protocol. The experiment demonstrates peer-to-peer and star-topology data transmission where slave nodes collect real-time sensor data and relay it to the master node. The master aggregates and streams the telemetry to a connected PC for live monitoring and analysis. For demonstration, each slave carries a DHT11 temperature and humidity sensor, and the design allows many other sensors to be added.

Theory

A telemetry mesh network is a distributed sensing system in which multiple nodes collect physical-world data and transmit it wirelessly to a central aggregation point. In this experiment, the ESP-NOW protocol is used to establish direct device-to-device communication without the overhead of a traditional Wi-Fi access point. ESP-NOW is a connectionless, low-latency protocol developed by Espressif for ESP32 and ESP8266 devices. Because both chip families support it, an ESP8266 master can communicate directly with ESP32 slaves, which makes this a heterogeneous network.

ESP-NOW operates at the data link layer (Layer 2 of the OSI model) and supports unicast and broadcast transmissions with a maximum payload of 250 bytes per packet. Because it bypasses the TCP/IP stack, it achieves significantly lower latency compared to standard Wi-Fi communication, making it well suited for real-time sensor telemetry.

Network Topology

The network follows a star topology with the NodeMCU ESP8266 master at the centre. Each ESP32 slave node transmits to the master using the master's unique MAC address. Slaves transmit structured data packets to the master at fixed intervals using the esp_now_send() API. The master, operating in the ESP-NOW slave/combo role, receives packets via a registered callback (onDataReceived) and immediately forwards the parsed data over a USB Serial link to the PC.

Data Packet Structure

Each transmission carries a fixed-size C struct containing the slave node ID, sensor readings (temperature and humidity), a validity flag, and a millisecond timestamp. The struct is declared with the packed attribute, so the ESP32 and ESP8266 interpret it with an identical byte layout. A fixed struct ensures deterministic packet size and simplifies parsing on both the master and the PC dashboard.

PC Dashboard

A Python script running on the connected PC reads the JSON-formatted serial output from the master node and renders a live terminal table using ANSI escape codes. The dashboard displays per-node temperature, humidity, validity status, and the time elapsed since the last update for each active node.

Requirements

Hardware

- 1 x NodeMCU ESP8266 (ESP-12E) development board (master node)
- 2 x ESP32 development boards (slave nodes)
- DHT11 temperature and humidity sensor (one per slave, used for demonstration)
- Optional sensors for future expansion (not used in this experiment): BMP280, MQ-series, soil moisture, LDR
- Jumper wires and breadboards
- USB cables for programming and master serial output
- 5V power supply for slave nodes

Software

- Arduino IDE with ESP32 and ESP8266 board support packages
- Master libraries: ESP8266WiFi.h, espnow.h
- Slave libraries: esp_now.h, WiFi.h, Wire.h, LiquidCrystal_I2C.h, DHT.h
- Python 3.x with pyserial library (pip install pyserial)
- VS Code (recommended) or any serial terminal for PC dashboard

Block Diagram Description

The Telemetry Mesh system is divided into three functional layers:

Sensor Layer (Slave Nodes)

Each slave ESP32 is connected to a single DHT11 sensor for temperature and humidity measurement. In this implementation, both slave nodes use only the DHT11 sensor, and no other sensors are connected. The slave reads sensor data at a fixed interval, packs the readings into a structured data packet, and transmits it to the master using ESP-NOW. The slave's node ID is configured at compile time via a preprocessor define (#define SLAVE_ID). Although the experiment uses only the DHT11 for demonstration, the design is not limited to it. The ESP32 has many free GPIO, ADC and I2C pins, so other sensors, such as pressure, gas, soil moisture or light sensors, can be added to any slave in future by adding extra fields to the packet struct.

Aggregation Layer (Master Node)

The NodeMCU ESP8266 master initialises in Wi-Fi station mode (required for ESP-NOW), sets its ESP-NOW role, and registers a receive callback. Upon receipt of a valid packet from any slave, it immediately serialises the data as a JSON string and writes it to the USB Serial port at 115200 baud. It also broadcasts a heartbeat message every five seconds listing all currently active node IDs.

Monitoring Layer (PC Dashboard)

A Python script opens the serial port, parses incoming JSON lines, and maintains a dictionary of the latest reading per node. The terminal is refreshed after every new packet, displaying a formatted table with node ID, temperature, humidity, and time since last update.

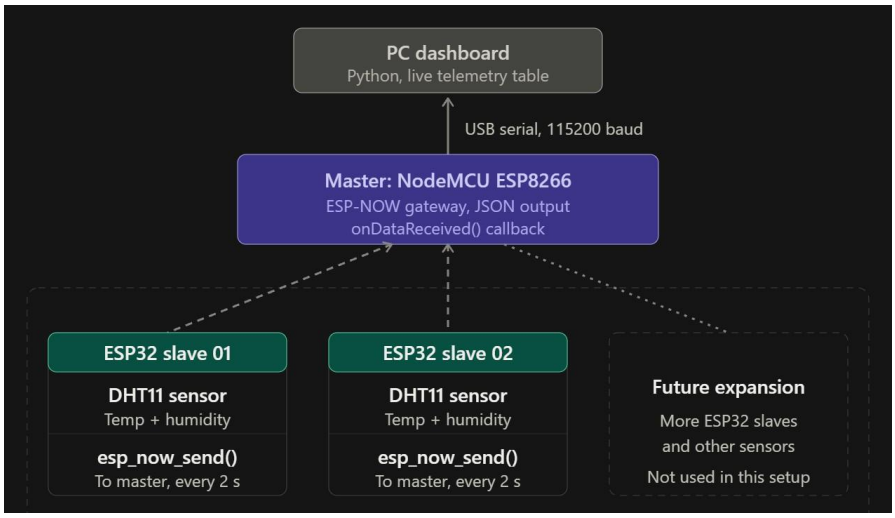


Fig. 1: Block diagram of the ESP-NOW telemetry mesh network with ESP8266 master and two ESP32 slaves

Design Calculations and Signal Analysis

Transmission Interval and Latency

Each slave transmits once every 2000 ms (configurable via SEND_INTERVAL). This interval also suits the DHT11, which can be sampled at most once per second. ESP-NOW unicast delivery latency is typically under 5 ms for line-of-sight distances up to 200 m indoors. Total end-to-end latency from sensor reading to PC display is therefore dominated by the sampling interval and USB Serial buffering, not the wireless link.

Packet Size
The SensorPacket struct contains: nodeID (1 byte) + temperature (4 bytes, float) + humidity (4 bytes, float) + timestamp (4 bytes, uint32_t) + valid flag (1 byte) = 14 bytes total. This is well within the 250-byte ESP-NOW payload limit, leaving headroom for future sensor additions.

MAC Address Registration
ESP-NOW requires that the master's MAC address be hardcoded into each slave before upload. The NodeMCU ESP8266 master prints its station MAC address over Serial on boot (as a JSON event message), which is then copied into the masterMAC[] array in each ESP32 slave sketch. Each slave must also be assigned a unique SLAVE_ID integer (1, 2, 3, ...) to identify its packets at the master.

Channel Alignment
Both master and slaves must operate on the same Wi-Fi channel. Since the ESP32 slaves register the master as a peer with channel = 0 (automatic), the protocol selects the active channel of the Wi-Fi interface. All nodes are kept in WIFI_STA mode without associating to an AP, ensuring they share the default channel 1.

DHT11 Sensor Specifications

The DHT11 measures temperature from 0 to 50 °C with ±2 °C accuracy and relative humidity from 20 to 90 % with ±5 % accuracy. It outputs a single-wire digital signal and operates at 3.3 V to 5 V, so it connects directly to the ESP32 without level shifting.

Hardware Implementation

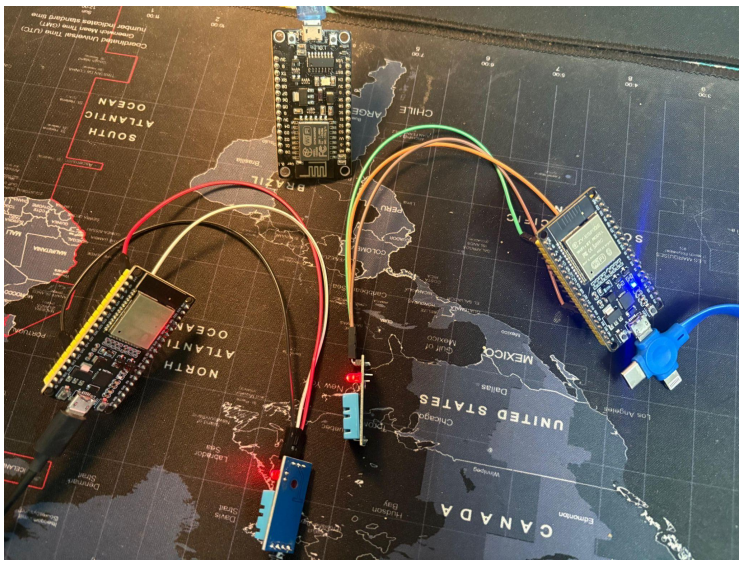


Fig. 2 : Complete hardware prototype with NodeMCU ESP8266 master and two ESP32 slaves with DHT11 sensors

Observations / Results

The following system behaviours were recorded during the experiment:

Parameter / Event	Observation
ESP-NOW Initialisation	NodeMCU ESP8266 master and both ESP32 slaves initialised successfully; master MAC printed on Serial boot
Slave-to-Master Transmission	Packets from the ESP32 slaves received by the ESP8266 master within 5 ms; callback triggered reliably every 2 s
Master Serial Output	Temperature and humidity updated every 2 s; node ID and uptime shown on second line
PC Dashboard	Live table refreshed on each incoming packet; all active nodes listed with readings
Node Added Mid-Run	New slave powered on mid-session; appeared in dashboard within one transmission cycle
Sensor Fault Simulation	DHT11 disconnected; valid flag set to false; dashboard displayed ERR status for that node
Range Test	Reliable packet delivery observed up to approximately 15 m through one concrete wall

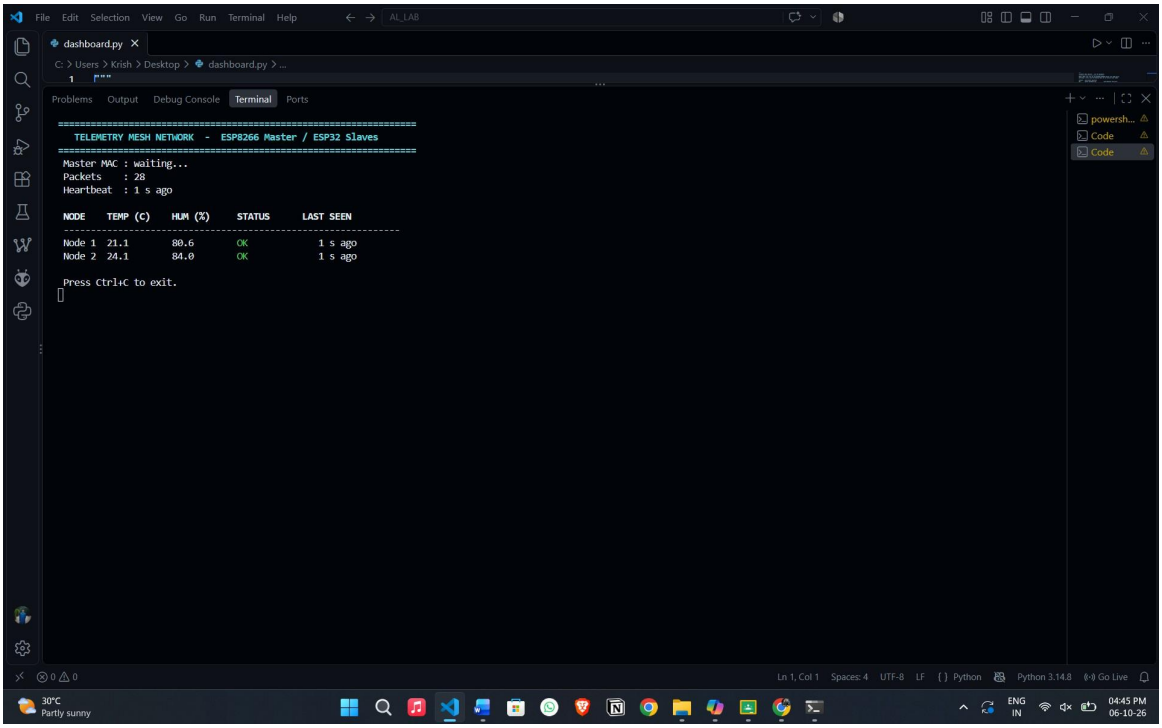


Fig. 3 : Live PC dashboard showing temperature and humidity from Node 1 and Node 2

Discussion of Results

- The experiment successfully demonstrated a functional wireless telemetry mesh using ESP-NOW between a NodeMCU ESP8266 master and two ESP32 slaves. This confirmed that ESP-NOW works across different Espressif chip families. The connectionless nature of ESP-NOW allowed slave nodes to begin transmitting immediately on power-up without any association handshake, which was visually effective during the live demonstration. Each slave appeared in the PC dashboard within one 2-second cycle of being powered on.
- The use of a fixed, packed C struct for packet serialisation ensured that all data fields were transmitted and received identically on both the ESP32 and the ESP8266, without additional encoding overhead. The validity flag proved useful when a sensor fault was simulated; the master correctly propagated the ERR status to the dashboard rather than displaying stale or garbage values.
- The Python terminal dashboard rendered a clean, refreshing live table using ANSI clear-screen codes, which made the system behaviour immediately interpretable to an observer. The os.system workaround was required on Windows to avoid artefacts from the ANSI escape sequence method.

- The DHT11 was chosen for demonstration because of its low cost and simple interface. Its limited range and accuracy (±2 °C, ±5 % RH) are acceptable for a demonstration, and more accurate sensors such as the DHT22 or BME280 can replace it without changing the network design.
- A minor limitation observed was that if a slave node powers off, its entry remains in the dashboard table with an increasing 'last seen' counter rather than being removed. This could be addressed in future work by implementing a timeout-based node eviction policy on the master or the dashboard script.

Real-Time Applications / Industrial Significance

- Industrial IoT sensor networks for temperature, pressure, and gas monitoring
- Smart agriculture — distributed field sensors reporting to a central gateway
- Building automation systems for environmental monitoring across multiple rooms
- Wireless data acquisition in laboratory and research environments
- Asset tracking and condition monitoring in warehouses and logistics
- Disaster response systems requiring rapid, infrastructure-free sensor deployment

Conclusion

The Telemetry Mesh Network was successfully designed and implemented using a NodeMCU ESP8266 as the master and two ESP32 microcontrollers as slave nodes, communicating via the ESP-NOW protocol.The slave nodes reliably acquired temperature and humidity data from DHT11 sensors and transmitted structured packets to the master node at 2-second intervals. The master aggregated all incoming telemetry and streamed it as JSON over USB Serial to a Python-based PC dashboard, which rendered a live, auto-refreshing table of all active nodes.

The experiment demonstrated key concepts in wireless embedded communication, including MAC-address-based peer registration, interoperability between ESP8266 and ESP32 devices, fixed-struct packet serialisation, callback-driven receive handling, and real-time terminal visualisation. The system can be scaled by adding further ESP32 slave nodes, or by connecting additional sensors (pressure, gas, soil moisture, light and others) to existing slaves, with minimal configuration changes. This makes it a practical foundation for larger IoT telemetry applications.

The experiment demonstrated key concepts in wireless embedded communication, including MAC-address-based peer registration, fixed-struct packet serialisation, callback-driven receive handling, and real-time terminal visualisation. The system can be scaled by adding further slave nodes with minimal configuration changes, making it a practical foundation for larger IoT telemetry applications.

References

- Espressif Systems — ESP-NOW Protocol Documentation (docs.espressif.com)
- ESP8266 Non-OS SDK — ESP-NOW User Guide (Espressif Systems)
- DHT11 Datasheet — Aosong Electronics
- Arduino ESP32 Core Library Reference — esp_now.h API
- DHT Sensor Library — Adafruit Industries (github.com/adafruit/DHT-sensor-library)
- LiquidCrystal I2C Library — Frank de Brabander
- PySerial Documentation — pyserial.readthedocs.io
- Internet of Things: A Hands-On Approach — Bahga & Madisetti